

DBMS & SQL - Revision Notes

Lecture 1: Foundations of DBMS & Relational Database Design | Lecture 2: Introduction to SQL and SQL Commands

Database used in this course: PostgreSQL. Syntax below is PostgreSQL-first. Where MySQL or Oracle differ in a way that matters, it's flagged in a *Dialect note*.

How to use these notes: Every section closes with a **Quick Recall** box. Read the section, cover it, and try to reproduce the box from memory. The checkpoint questions from the slides appear in Part 10 with answers.

PART 1 - WHAT IS DATA?

1.1 The Zomato hook

Every time you open a food-delivery app, it is silently collecting data about you:

What it collects	Why
Your location	Where you are right now
Order history	What you've ordered before
Restaurant details	Menus, hours, delivery zones
Ratings & reviews	What others say about each place
Your preferences	Cuisines you keep returning to
Dish popularity	What's trending in your area

Stored data -> intelligent behaviour. Those observations power concrete features:

- **Nearby Restaurants** <- location data filtered by delivery radius
- **Top Picks** <- ranking driven by your past preferences and ratings
- **Personalized Recommendations** <- machine learning on order history + cuisine preference
- **Favourite Dishes** <- tracking repeat orders

The core idea of the whole course: *an application feels intelligent only because it stored observations first.*

1.2 Definition and the ladder

Data is a collection of **raw facts, figures, and observations** that have not yet been processed or given meaning.

The four-rung ladder from the slides:

None

1. Observations -> Raw facts collected from the real world
2. Data -> Recorded observations stored systematically
3. Information -> Data placed in context that gives it meaning
4. Insight -> Information analyzed to drive decisions

Organizations collect and store data first - analysis comes later. This is why storage design (schema) precedes analytics.

Quick Recall 1: Data = raw, unprocessed facts. Ladder = Observations -> Data -> Information -> Insight. Store first, analyse later.

PART 2 - CATEGORIES OF DATA

Why this matters: *how data is organized determines how it should be stored.* Each category gets its own analogy - learn the analogy and the definition follows.

2.1 Structured Data - *The School Records Room*

None

AISLE -> CUPBOARD -> DRAWER -> FILE
Class 10 Section A Mathematics Half-Yearly Exam

Why it works: every student in a class studies the *same subjects* and appears for the *same examinations*. The answer sheets follow a predictable, identical pattern across all students, so this predefined arrangement makes locating any student's paper extremely efficient.

Definition: Data organized in a **fixed, predefined format** with rows and columns. Every record follows the same schema.

Stored in: Relational databases, data warehouses.

Table representation

cust_id	name	email	city	is_prime
101	Rahul	rahul@mail.com	Pune	TRUE
102	Sara	sara@mail.com	Mumbai	FALSE
103	Arjun	arjun@mail.com	Delhi	TRUE

Every row follows the same structure; every column has a defined data type. **This predictability is what makes structured data fast to query and easy to validate.**

2.2 Semi-Structured Data - *The University Locker*

In a university, every student's journey is unique:

- Different streams (Arts, Science, Tech)
- Different electives & specializations
- Unique internships, projects, hackathons
- Individual certifications & skill sets

The locker analogy: you can't organize university documents the way you organized school papers - students don't share identical subjects. Instead, assign each student a **personal locker** containing documents unique to their journey. *Structure exists* (student name, enrollment ID) *but the contents vary widely*.

Flexibility accommodates diversity - but **reduces search efficiency** compared to the structured records room.

Definition: Data with **some organizational properties but no rigid schema**. Tags and markers provide partial structure.

Stored in: Document stores (MongoDB), XML/JSON files.

JSON representation - the same customer

```
JSON
{
  "cust_id": 101,
  "name": "Rahul",
  "email": "rahul@mail.com",
  "city": "Pune",
  "is_prime": true,
  "recent_orders": ["Biryani", "Pizza"]    <- nested array
}
```

Tags provide structure, but each document can have **different fields and nesting** - no rigid schema required.

2.3 Unstructured Data - *The Talent Hunt*

A college talent hunt invites all kinds of submissions: music recordings, paintings, sculptures, video performances, portfolios, photographs.

Every submission is **completely different in format, size, and type**. There is no predefined schema - you can only **tag each submission with metadata** (student name, category, date) and store it. Searching relies on labels, not structure - making retrieval significantly less efficient.

Definition: Data with **no predefined format or organization**. Cannot be stored in rows and columns.

Stored in: Data lakes, object storage (S3), file systems.

Real examples: product images (.jpg, .png), call recordings (.mp3), scanned invoices (.pdf), social media posts (text), surveillance video (.mp4), email attachments (mixed).

Data Lake - a massive storage system designed to hold unstructured and semi-structured data **in its raw format** - like one giant room where all talent hunt submissions are dumped with tags. *Examples: AWS S3, Apache Iceberg.*

2.4 The three side by side

	Structured	Semi-Structured	Unstructured
Analogy	School records room	University locker	Talent hunt
Schema	Fixed, predefined	Partial (tags/markers)	None
Form	Rows & columns	Documents, nested fields	Files, media
Searchability	Fastest	Moderate	Slowest (metadata only)
Stored in	Relational DBs, data warehouses	MongoDB, XML/JSON	Data lakes, S3, file systems

2.5 The core trade-off: adding a new attribute

Scenario: only Rahul has a `loyalty_points` attribute.

Relational table - a new column is added for **ALL** rows:

cust_id	name	loyalty_points
101	Rahul	450
102	Sara	NULL
103	Arjun	NULL

Sara and Arjun get `NULL` - **wasted storage for data they don't have**, plus a schema migration.

JSON document - only one document changes:

```
JSON
// Only Rahul's document changes:
{ "cust_id": 101, "name": "Rahul", "loyalty_points": 450 }  <- added
// Sara & Arjun: unchanged
```

No wasted storage, no schema migration.

Rigid schema vs flexible documents - each has trade-offs. This single slide is the strongest exam question in Lecture 1. Rigid buys speed and validation; flexible buys adaptability.

Quick Recall 2: Structured = school records room (fixed schema, fastest). Semi-structured = university locker (tags, varies per record). Unstructured = talent hunt (no schema, metadata only, data lake). Adding one attribute: table -> NULLs for everyone; JSON -> one document changes.

PART 3 - TYPES OF DATABASE MANAGEMENT SYSTEMS

3.1 Relational Databases

Definition: A database that stores data in **tables** - a fixed set of columns decided **before any data arrives**, and rows that all follow it. **Two tables are connected by repeating a shared value in both.**

The data it holds: *Structured data*. Same fields for every record, known in advance - the **school records room**. Rows and columns fit it exactly, which is why retrieval stays fast and every value can be validated as it goes in.

What it does NOT fit: Records where each one carries different fields, and files with no rows or columns at all - a JSON document with extra keys, a video, a scanned invoice. *Forcing those into a table means empty columns, or no place to put them.*

In practice

System	Note
PostgreSQL	Open source - used through this course
MySQL	Behind a large share of web applications
Oracle	Banking and enterprise systems
SQL Server	Microsoft's enterprise database
SQLite	Embedded inside phones and browsers

3.2 Non-Relational (NoSQL) Databases

Definition: A database that **does not force every record into one fixed table structure**. Each record carries its own fields, and the structure can differ from one record to the next.

The data it holds - 1: Semi-structured data. The *university locker*. Tags give partial structure, but the fields vary per record - adding **loyalty_points** to one record leaves the others untouched.

The data it holds - 2: Unstructured data. The *talent hunt*. Images, audio, video and documents are kept whole and found through the **labels attached to them**, not through rows and columns.

In practice

System	What it does
MongoDB	Stores JSON-style documents, one per record
Redis	Keeps a value against a key, in memory
Cassandra	Each record may carry its own set of columns

Neo4j Stores things and the links between them

Non-relational is a **family of stores**, not one thing.

3.3 Relational vs Non-Relational - summary

	Relational	Non-Relational
Structure	Fixed tables, fixed columns	Flexible; varies per record
Schema decided	Before data arrives	Not required upfront
Links between records	Shared value (foreign key)	Embedding, references, or graph edges
Best for	Structured data	Semi-structured & unstructured
Language	SQL	Varies (APIs, query DSLs)
Examples	PostgreSQL, MySQL, Oracle, SQL Server, SQLite	MongoDB, Redis, Cassandra, Neo4j

Quick Recall 3: Relational = fixed tables, shared values link them, PostgreSQL/MySQL/Oracle/SQL Server/SQLite. Non-relational = per-record structure, MongoDB (documents) / Redis (key-value) / Cassandra (wide column) / Neo4j (graph).

PART 4 - WHY DBMS? ADVANTAGES OVER FILE-BASED STORAGE

The slides frame each advantage as **File-Based Problem -> DBMS Solution -> Result**. Reproduce that three-part structure in the exam and you cannot lose marks.

1. Eliminating Data Redundancy

- **File-Based Problem:** Student phone numbers might be duplicated in `attendance.csv`, `fees.xlsx`, and `contacts.txt` - each potentially different.
- **DBMS Solution:** Phone number is stored **once** in a `Students` table. `Attendance` and `Fees` tables reference the student via `student_id`.
- **Result:** Update once -> reflected everywhere. **No conflicting copies.**

2. Ensuring Data Consistency

- **Problem:** If a student's name is 'Rahul' in one file and 'Rahul S.' in another - which is correct? Files have **no mechanism to enforce uniformity**.
- **Solution: Constraints** ensure: every student must have a name; email must be unique; age must be positive.
- **Result: Invalid data is rejected before it enters the system.**

3. Efficient Data Access & Querying

- **Problem:** Finding all students from Pune in a text file? You must read **every line**. 1 million records = 1 million comparisons.
- **Solution:** The DBMS uses an **index** to jump directly to matching records.
- **Result: Seconds instead of minutes** - even on millions of rows.

4. Concurrent Access Control

- **Problem:** Two staff open **fees.xlsx** at the same time. Both edit row 42. **The last save wins** - the other's changes are silently lost.
- **Solution:** DBMS **locks the specific row** being edited. Other users can still access all other rows. Conflicts are detected and resolved automatically.
- **Result: Thousands of users** can work on the same database simultaneously.

5. Data Security & Access Control

- **Problem:** If **fees.xlsx** is on a shared drive, anyone with folder access can view, edit, or delete any record. **No granularity**.
- **Solution:** Grant the accountant *entry* on fees, but only *retrieval* on student records. Admin gets full access; students get read-only on their own records.
- **Result: Fine-grained permissions** at table, column, and even row level.

6. Backup & Recovery

- **Problem:** A power outage while saving a large CSV? The file may be **half-written and unrecoverable**. No automatic backup mechanism.
- **Solution:** DBMS writes changes to a **transaction log before applying them**. After a crash, it **replays permanent transactions and rolls back incomplete ones**.
- **Result:** Data survives hardware failures, power outages, and software crashes.

7. Data Integrity via Constraints

- **Problem:** An order references **customer_id 999**, but that customer was deleted from the file. The order now points to nobody - an **orphan record**.
- **Solution:** The DBMS **refuses to delete a customer who has existing orders**.
- **Result:** Broken relationships and orphan records become **impossible**.

Orphan record - a record referencing a parent that no longer exists (e.g., an order for a deleted customer). Foreign keys prevent this. *This term appears in Checkpoint 2 - know the one-line definition.*

Quick Recall 4 - the seven advantages in order: R-C-A-C-S-B-I ->
Redundancy eliminated | Consistency enforced | Access efficient | Concurrency controlled | Security granular | Backup & recovery | Integrity via constraints.

PART 5 - SCHEMA: THE BLUEPRINT

Structure first, data second.

5.1 What is a Schema?

A **schema** is the **logical blueprint that defines the structure of a database before any data is inserted**. It defines **four** things:

Component	Definition	Example
Entities	Real-world objects represented as tables	Customer, Product, Order
Attributes	Properties of each entity stored as columns	name, email, price, quantity
Relationships	How tables connect to each other	Orders belong to Customers via <code>cust_id</code>
Constraints	Rules that enforce data integrity	NOT NULL, UNIQUE, PRIMARY KEY, FOREIGN KEY

The mapping to remember:

```
None
Real world  ->  Database
-----
Entity      ->  Table
Attribute   ->  Column
Instance    ->  Row
Relationship->  Shared value (foreign key)
```

5.2 Data Types: What a Column May Hold

Every column **declares a datatype when the table is created**. It fixes:

1. which values are accepted,
2. how much space each one takes,
3. which operations make sense on the column.

Datatype	Holds	Column	Value
INT	Whole numbers, positive or negative	age	27
DECIMAL(10,2)	Exact numbers with a fixed number of decimals (money)	amount	450.00
VARCHAR(n)	Text up to n characters, stored at its real length	name	'Rahul'
CHAR(n)	Text of exactly n characters, padded if shorter	country_code	'IN'
BOOLEAN	One of two states, true or false	is_prime	TRUE
DATE	A calendar date, comparable and sortable	dob	2004-07-19
TIMESTAMP	A date together with the time of day	placed_at	2026-08-06 14:32

Why it matters (exam answer): Store `age` as text and `'9'` sorts *after* `'27'`, averages become impossible, and `'twenty'` is accepted without complaint. **The datatype is the first rule a column enforces - the rules that follow build on it.**

5.3 Constraints, One at a Time

NOT NULL

Ensures a column **can never be left empty**. Every row must supply a value for it - a missing value is rejected at insert time.

SQL

```
CREATE TABLE Customers ( name VARCHAR(50) NOT NULL );
```

Rejected: `INSERT INTO Customers(name) VALUES (NULL);`

UNIQUE

Ensures every value in a column is **distinct across all rows** - no two rows may share the same value. (**NULLs are still permitted.**)

SQL

```
CREATE TABLE Customers ( email VARCHAR(100) UNIQUE );
```

Rejected: a second row with `email = 'rahul@mail.com'`

Note: NOT NULL and UNIQUE are **independent** - a column can be one, both, or neither.

CHECK

Enforces a **condition every value must satisfy**. Any row that violates the condition is rejected - keeps invalid data out at write time.

SQL

```
CREATE TABLE Customers ( age INT CHECK (age >= 18) );
```

Rejected: `INSERT INTO Customers(age) VALUES (15);`

DEFAULT

Supplies an **automatic value** for a column when the INSERT statement doesn't provide one - data is never left ambiguous.

SQL

```
CREATE TABLE Customers ( is_prime BOOLEAN DEFAULT FALSE );
```

Result: INSERT with no `is_prime` given -> stored as `FALSE`

Together, **NOT NULL**, **UNIQUE**, **CHECK** and **DEFAULT** constrain individual columns. **PRIMARY KEY** and **FOREIGN KEY** govern whole rows and relationships between tables.

5.4 Deep Dive: PRIMARY KEY

The column (or set of columns) that uniquely identifies every row in a table.

Four properties:

1. **Unique** - no two rows can share the same value.
2. **Not Null** - every row must have a value; identity can't be unknown.

3. **One per table** - exactly one primary key, though it **can span multiple columns (composite key)**.
4. **Immutable in spirit** - chosen so its value rarely, if ever, needs to change.

SQL

```
CREATE TABLE Customers (
  cust_id INT PRIMARY KEY,
  name    VARCHAR(50) NOT NULL
);
```

cust_id	name
101	Rahul
102	Sara

Rejected: inserting `cust_id = 101` again (duplicate), or `cust_id = NULL` (missing identity).

Why it matters: every other table refers to a row **by its primary key** - it's the anchor a foreign key points to.

5.5 Deep Dive: FOREIGN KEY

A column in one table that references the primary key of another table - the link that connects two tables.

- **Referential integrity** - every `cust_id` in `Orders` must already exist as a `cust_id` in `Customers`.
- **Prevents orphan rows** - a child row can never point to a parent that doesn't exist.

SQL

```
CREATE TABLE Orders (
  order_id INT PRIMARY KEY,
  cust_id  INT,
  FOREIGN KEY (cust_id) REFERENCES Customers(cust_id)
);
```

Customers:
cust_id

Orders: order_id cust_id

101	-> 501	101
102	-> 502	102

Rejected: `INSERT INTO Orders(order_id, cust_id) VALUES (503, 999);` - no customer 999 exists, so the order is orphaned and blocked.

Why it matters: foreign keys are how relational databases **model real relationships** - a customer having many orders - **without duplicating data**.

5.6 Super Key -> Candidate Key -> Primary Key

Three related ideas about what can uniquely identify a row - **from broadest to most specific**.

Term	Definition
Superkey	Any set of one or more columns that uniquely identifies a row. May contain extra, unnecessary columns.
Candidate Key	A minimal superkey - remove any column and it stops being unique. A table can have several.
Primary Key	The one candidate key the designer picks as the table's main identifier. The rest remain candidates, enforceable via <code>UNIQUE</code> .

Worked example - `Customers(cust_id, email, phone, name, city)`

Column set	Superkey ?	Candidate key?	Primary key?
<code>{cust_id}</code>	Yes	Yes	**Yes **
<code>{email}</code>	Yes	Yes	No
<code>{phone}</code>	Yes	Yes	No
<code>{cust_id, name}</code>	Yes	No - name is redundant	No

Every primary key is a candidate key, and every candidate key is a superkey - but not the reverse.

5.7 Relationships (how tables connect)

A relationship in a relational database is implemented by **repeating a shared value in both tables** - the foreign key.

Cardinality	Meaning	Example	How it's built
One-to-One (1:1)	Each A relates to at most one B	Customer <-> Loyalty card	FK + UNIQUE on the FK column
One-to-Many (1:N)	One A relates to many B	One Customer -> many Orders	FK on the "many" side (Orders.cust_id)
Many-to-Many (M:N)	Many A relate to many B	Students <-> Courses	A third (junction) table holding both FKs

Why M:N needs a third table: you cannot store a list inside a single column of a relational table. So you create:

SQL

```
CREATE TABLE Enrollment (  
    student_id INT,  
    course_id INT,  
    grade CHAR(2),  
    PRIMARY KEY (student_id, course_id),  
    composite key  
    FOREIGN KEY (student_id) REFERENCES Students(student_id),  
    FOREIGN KEY (course_id) REFERENCES Courses(course_id)  
);
```

Quick Recall 5: Schema = blueprint defined *before* data, with **four** parts: entities (tables), attributes (columns), relationships (foreign keys), constraints (rules). Column-level constraints: NOT NULL, UNIQUE, CHECK, DEFAULT. Row/table-level: PRIMARY KEY, FOREIGN KEY. Superkey contains Candidate key contains Primary key.

PART 6 - INTRODUCTION TO SQL

6.1 The big question

Databases hold billions of records. How do humans and applications actually talk to them?

- How does Instagram fetch your feed in milliseconds?
- How does Amazon update prices for millions of users?

- How does a company decide who can access sensitive data?
- How can a bank undo a failed transaction?

The answer to all four is **SQL**.

6.2 What SQL is

SQL (Structured Query Language) is the **standard language used to communicate with relational databases**.

How the conversation works:

None

```
User / Application -- SELECT name FROM students; -->
Database
User / Application <----- rows returned -----
Database
```

6.3 The library analogy

A database is a **library**. SQL is the **librarian's language**.

Action	In the library	In the database
Organize	Arranging books on shelves	Designing tables and schemas
Search	Finding a specific book fast	Retrieving exact records
Modify	Replacing a damaged book	Updating or deleting records
Secure	Reference-only sections	Controlling who can access what

6.4 Systems that use SQL

System	Note
MySQL	Widely used for web applications (e.g. WordPress)
PostgreSQL	Open-source, known for advanced features and SQL standards
Oracle	Enterprise database - banks, airlines, governments
SQL Server	Enterprise RDBMS integrated with the Microsoft ecosystem

Although these databases have **different features**, they all use **SQL** as their standard language for managing and querying data.

6.5 Categorization of SQL Commands

One language, four job descriptions. As databases grew larger, SQL commands were grouped by the *kind of task* they perform - just like running a university.

Category	Full form	University analogy	Commands
DDL	Data Definition Language	Constructing classrooms	CREATE
DML	Data Manipulation Language	Managing student records	INSERT
TCL	Transaction Control Language	Undoing mistakes safely	COMMIT
DCL	Data Control Language	Deciding who gets access	GRANT

Note on SELECT: in this course **SELECT** is taught under **DML**. (Some textbooks split it into a fifth category, **DQL - Data Query Language**. If you use that term in an exam, mention that this course groups it with DML.)

One structural difference worth knowing: DDL commands **auto-commit** - they take effect immediately and cannot be undone with **ROLLBACK**. DML commands do **not** auto-commit, which is exactly why TCL exists. *This is why **DELETE** can be rolled back but **TRUNCATE** cannot.*

Quick Recall 6: SQL = standard language for relational databases. Library analogy: organize / search / modify / secure. Four categories - **DDL** (structure), **DML** (data), **TCL** (transactions), **DCL** (permissions).

PART 7 - DDL: DATA DEFINITION LANGUAGE

DDL commands define and manage database structures - *before any data can live inside a table, the table itself must exist.*

They primarily affect: **Databases | Tables | Schema definitions**

The running example from the slides is a **students** table. We'll follow it all the way through.

7.1 CREATE - *building the classroom*

The principal walks in: *"We need somewhere to store every student's ID and name before classes begin."*

CREATE builds a brand-new database or table from a schema definition - the direct continuation of table design, now actually running. **Schema design becomes executable.**

Syntax

```
SQL
CREATE TABLE table_name (
    column1 datatype [constraints],
    column2 datatype [constraints],
    ...
    [table-level constraints]
);
```

The slide example

```
SQL
CREATE TABLE students (
    id    INT PRIMARY KEY,
    name  VARCHAR(50)
);
```

BEFORE

No table yet `id | name` (empty table, structure ready)

AFTER

Creating a database

```
SQL
CREATE DATABASE college_db;
```

Dialect note: MySQL and PostgreSQL both support `CREATE DATABASE`; MySQL also allows `CREATE DATABASE IF NOT EXISTS college_db`;

A fuller example, using every constraint from Part 5

SQL

```
CREATE TABLE customers (  
  cust_id INT PRIMARY KEY,  
  name VARCHAR(50) NOT NULL,  
  email VARCHAR(100) UNIQUE,  
  age INT CHECK (age >= 18),  
  city VARCHAR(30) DEFAULT 'Pune',  
  is_prime BOOLEAN DEFAULT FALSE  
);  
  
CREATE TABLE orders (  
  order_id INT PRIMARY KEY,  
  cust_id INT,  
  amount DECIMAL(10,2),  
  placed_at TIMESTAMP,  
  FOREIGN KEY (cust_id) REFERENCES customers(cust_id)  
);
```

Order matters. Create **customers** before **orders** - a foreign key cannot reference a table that doesn't exist yet.

7.2 ALTER - renovating the classroom

The principal walks in: "We also need to track each student's branch. Add that column without losing any existing data."

SQL

```
ALTER TABLE students  
ADD COLUMN branch VARCHAR(20);
```

BEFORE

id	name
1	Amit

AFTER

id	name	branch
1	Amit	CSE

One command, many targets. ALTER modifies an existing structure - add, rename, or drop columns, change data types, or adjust constraints like PRIMARY KEY - all without rebuilding the table.

All functions and operations of ALTER

A. ADD a column

SQL

```
ALTER TABLE students ADD COLUMN branch VARCHAR(20);

-- With a constraint / default
ALTER TABLE students ADD COLUMN backlogs INT DEFAULT 0;
```

Existing rows get **NULL** in the new column - or the **DEFAULT** value if one is supplied.

B. ADD multiple columns

SQL

```
-- PostgreSQL
ALTER TABLE students
  ADD COLUMN email VARCHAR(100),
  ADD COLUMN dob   DATE;

-- Oracle
ALTER TABLE students ADD (email VARCHAR2(100), dob DATE);
```

C. CHANGE a column's data type

SQL

```
-- PostgreSQL
ALTER TABLE students ALTER COLUMN name TYPE VARCHAR(100);

-- MySQL / Oracle
ALTER TABLE students MODIFY name VARCHAR(100);
```

Widening (**VARCHAR(50)** -> **VARCHAR(100)**) is safe. **Narrowing** may fail if existing data doesn't fit.

D. RENAME a column

SQL

```
ALTER TABLE students RENAME COLUMN branch TO department;
```

E. DROP a column

SQL

```
ALTER TABLE students DROP COLUMN branch;
```

This **permanently destroys that column's data in every row.**

F. RENAME the table

SQL

```
ALTER TABLE students RENAME TO student_master;
```

G. SET or DROP a DEFAULT

SQL

```
ALTER TABLE students ALTER COLUMN branch SET DEFAULT 'CSE';  
ALTER TABLE students ALTER COLUMN branch DROP DEFAULT;
```

H. SET or DROP NOT NULL

SQL

```
-- PostgreSQL  
ALTER TABLE students ALTER COLUMN name SET NOT NULL;  
ALTER TABLE students ALTER COLUMN name DROP NOT NULL;  
  
-- MySQL / Oracle (restate the type)  
ALTER TABLE students MODIFY name VARCHAR(50) NOT NULL;
```

I. ADD a constraint

SQL

```
ALTER TABLE students ADD CONSTRAINT pk_students PRIMARY KEY (id);  
ALTER TABLE students ADD CONSTRAINT uq_email UNIQUE (email);  
ALTER TABLE students ADD CONSTRAINT chk_age CHECK (age >=  
17);  
ALTER TABLE orders ADD CONSTRAINT fk_cust  
FOREIGN KEY (cust_id) REFERENCES customers(cust_id);
```

Adding a constraint **fails if existing rows already violate it** - clean the data first.

J. DROP a constraint

SQL

-- PostgreSQL / Oracle

```
ALTER TABLE students DROP CONSTRAINT chk_age;
```

-- MySQL specifics

```
ALTER TABLE students DROP PRIMARY KEY;
```

```
ALTER TABLE students DROP FOREIGN KEY fk_cust;
```

```
ALTER TABLE students DROP INDEX uq_email;
```

Always name your constraints (`CONSTRAINT chk_age CHECK (...)`).

Unnamed constraints get system-generated names that are painful to find when you need to drop them.

ALTER - master summary

Operation	PostgreSQL syntax
Add column	<code>ALTER TABLE t ADD COLUMN c type;</code>
Change type	<code>ALTER TABLE t ALTER COLUMN c TYPE newtype;</code>
Rename column	<code>ALTER TABLE t RENAME COLUMN old TO new;</code>
Drop column	<code>ALTER TABLE t DROP COLUMN c;</code>
Rename table	<code>ALTER TABLE t RENAME TO newname;</code>
Set default	<code>ALTER TABLE t ALTER COLUMN c SET DEFAULT v;</code>
Set NOT NULL	<code>ALTER TABLE t ALTER COLUMN c SET NOT NULL;</code>
Add constraint	<code>ALTER TABLE t ADD CONSTRAINT n TYPE (c);</code>
Drop constraint	<code>ALTER TABLE t DROP CONSTRAINT n;</code>

Why ALTER matters (Checkpoint answer): *Requirements change over time - ALTER lets you add, modify, or drop columns and constraints **without losing existing data or rebuilding the table.***

7.3 TRUNCATE - *emptying the bucket*

The principal walks in: "New semester - clear out last year's student roster, but keep the students table ready for the new batch."

SQL

```
TRUNCATE TABLE students;
```

BEFORE

1	Amit	CSE
2	Riya	ECE
3	Arjun	CSE

AFTER

0 rows - structure kept

Analogy: emptying a bucket. TRUNCATE removes **all records instantly** while keeping the table structure fully intact - like emptying a bucket **without losing the bucket itself. No filtering; it's all rows, every time.**

Key characteristics

- Category: **DDL** -> auto-commits -> **cannot be rolled back**
- **No WHERE clause possible** - all or nothing
- **Fast:** deallocates data pages in bulk instead of logging each row
- Resets auto-increment / identity counters
- Fails if another table's foreign key references this table

7.4 DROP - *throwing away the bucket*

The principal walks in: "This batch's program has ended. Remove the students table completely - structure and all."

SQL

```
DROP TABLE students;
```

BEFORE

id | name | branch (with rows)

AFTER

Table no longer exists

Analogy: throwing away the bucket. DROP completely removes a database object - **table, structure, and every row inside it - permanently**. Unlike TRUNCATE, **there is nothing left to reuse afterward**.

SQL

```
DROP TABLE students;  
DROP DATABASE college_db;  
DROP TABLE IF EXISTS students;      -- avoids an error if absent
```

DROP fails if another table references it by foreign key. Drop the child table first, or use `DROP TABLE customers CASCADE;` (PostgreSQL).

7.5 DELETE vs TRUNCATE vs DROP

This comparison has its own slide in Lecture 2 - expect it in the exam.

	DELETE	TRUNCATE	DROP
What it removes	Selected rows	ALL rows	The table itself
WHERE condition	Can use one	No filtering	No filtering
Speed	Slower (row-by-row)	Faster (bulk operation)	Fastest - structure gone too
Table structure	Stays	Stays	Is deleted
Category	DML	DDL	DDL
Rollback possible	Yes (before COMMIT)	No	No

Why is TRUNCATE faster than DELETE? DELETE logs each row removal individually; TRUNCATE deallocates the data pages in bulk. *(Memorise this sentence - it is the exact answer expected.)*

Memory hook: DELETE = rub out some writing. TRUNCATE = empty the bucket. DROP = throw away the bucket.

Quick Recall 7: DDL = CREATE | ALTER | TRUNCATE | DROP. Build -> renovate -> empty the bucket -> throw away the bucket. All four auto-commit; none can be rolled back.

PART 8 - DML: DATA MANIPULATION LANGUAGE

If DDL builds the structure, DML handles the contents - it's how we work with the data actually living inside tables.

INSERT - add new rows | **SELECT** - retrieve rows | **UPDATE** - modify existing rows | **DELETE** - remove selected rows

8.1 INSERT - *enrolling a new student*

The principal walks in: "A new student just joined CSE. Add their record to the roll."

SQL

```
INSERT INTO students (id, name, branch)
VALUES (7, 'Kabir', 'CSE');
```

BEFORE			AFTER		
5	Ankit	CSE	5	Ankit	CSE
6	Asha	ECE	6	Asha	ECE
			7	Kabir	CSE

Adding rows, one at a time (or many). INSERT adds new rows of data into a table. You can insert a single row, or list several **VALUES** groups to add many rows in one statement.

Form 1 - With column names (recommended)

SQL

```
INSERT INTO students (id, name, branch)
VALUES (7, 'Kabir', 'CSE');
```

Form 2 - WITHOUT mentioning the sequence of attributes

SQL

```
INSERT INTO students
VALUES (8, 'Meera', 'ECE');
```

Five rules you **MUST** follow when you omit the column list:

1. You must supply a value for **every column** in the table.

2. Values must be in the **exact order the columns were defined** in `CREATE TABLE`.
3. Data types must match **positionally**.
4. Use `NULL` explicitly for any column you want left empty (if it allows NULL).
5. `DEFAULT` values are **not** applied automatically - write the keyword `DEFAULT` in that position if you want it.

SQL

```
-- customers column order: cust_id, name, email, age, city,
is_prime
```

```
INSERT INTO customers
VALUES (104, 'Neha', 'neha@mail.com', 22, 'Nashik', TRUE);
```

```
-- Leaving a value unknown
INSERT INTO customers
VALUES (105, 'Om', NULL, 25, 'Pune', FALSE);
```

```
-- Asking for the column's DEFAULT
INSERT INTO customers
VALUES (106, 'Ira', 'ira@mail.com', 21, DEFAULT, DEFAULT);
```

Why this form is risky. The database matches values **by position, not by meaning**. If someone later runs `ALTER TABLE ... ADD COLUMN`, or the columns are ever reordered, every positional INSERT breaks - or worse, **silently writes values into the wrong columns**. Naming the columns is self-documenting and survives schema change. *Use the short form only for quick practice; name your columns in real code.*

Comparison

	With column list	Without column list
Must list every column?	No - partial insert allowed	Yes
Order matters?	No - any order you like	Yes - defined order exactly
Survives <code>ALTER TABLE ADD</code> ?	Yes	Breaks
Uses <code>DEFAULT</code> automatically?	For omitted columns	Must write <code>DEFAULT</code>

Form 3 - Partial insert (only some columns)

SQL

```
INSERT INTO students (id, name)
VALUES (9, 'Farhan');
```

Omitted columns get their **DEFAULT**, or **NULL** if none. **Fails** if an omitted column is **NOT NULL** with no default.

Form 4 - Multiple rows in one statement

SQL

```
INSERT INTO students (id, name, branch)
VALUES
  (10, 'Tara', 'CSE'),
  (11, 'Yash', 'ECE'),
  (12, 'Riya S', 'ME');
```

Common INSERT errors

Error message	Cause
<i>column count doesn't match value count</i>	Omitted the column list but gave the wrong number of values
<i>duplicate key value violates unique constraint</i>	Primary key or UNIQUE value already exists
<i>insert or update violates foreign key constraint</i>	The referenced parent row doesn't exist
<i>null value in column "name" violates not-null constraint</i>	NOT NULL was breached
<i>new row violates check constraint</i>	e.g. <code>age = 15</code> when <code>CHECK (age >= 18)</code>
<i>value too long for type character varying(n)</i>	String exceeds <code>VARCHAR(n)</code>

8.2 SELECT - asking the database a question

The principal walks in: "Just show me the whole list - every student you have."

SQL

```
SELECT *
FROM students;
```

The most-used command in the industry. SELECT retrieves data - it changes nothing. It's the command you'll write more than any other.

Basic forms

```
SQL
SELECT * FROM students;           -- every column, every
row
SELECT name, branch FROM students; -- chosen columns only
SELECT DISTINCT branch FROM students; -- unique values only
SELECT name AS student_name FROM students; -- column alias
```

SELECT with a simple WHERE

WHERE filters rows. Only rows for which the condition is **TRUE** are returned.

```
SQL
SELECT column_list
FROM table_name
WHERE condition;
```

```
SQL
SELECT * FROM students WHERE branch = 'CSE';
SELECT name FROM students WHERE id = 3;
SELECT * FROM customers WHERE age > 21;
SELECT name, city FROM customers WHERE city = 'Pune';
```

Comparison operators

Operator	Meaning	Example
=	Equal to	WHERE branch = 'CSE'
<> or !=	Not equal to	WHERE branch <> 'ECE'
>	Greater than	WHERE age > 21
<	Less than	WHERE age < 18
>=	Greater than or equal	WHERE amount >= 500

`<=` Less than or equal `WHERE age <= 25`

Quotes rule: text and dates go in **single quotes** (`'CSE'`, `'2024-07-19'`);
numbers and booleans do not (`21`, `TRUE`).

Logical operators

SQL

```
SELECT * FROM customers WHERE city = 'Pune' AND is_prime = TRUE;
SELECT * FROM customers WHERE city = 'Pune' OR city = 'Mumbai';
SELECT * FROM customers WHERE NOT city = 'Delhi';
```

-- Bracket when mixing AND with OR

```
SELECT * FROM customers
WHERE (city = 'Pune' OR city = 'Mumbai') AND age > 21;
```

Precedence: `NOT` -> `AND` -> `OR`. **When in doubt, use brackets.**

Useful extras

SQL

```
SELECT * FROM customers WHERE age BETWEEN 18 AND 25;           --
inclusive both ends
SELECT * FROM customers WHERE city IN ('Pune', 'Mumbai', 'Delhi');
SELECT * FROM customers WHERE email IS NULL;                   -- ONLY
way to test NULL
SELECT * FROM customers WHERE name LIKE 'R%';                  --
starts with R
SELECT * FROM students ORDER BY name ASC;                      -- sort
result
```

WHERE email = NULL always returns nothing. NULL means *unknown* - it is not equal to anything, not even another NULL. **Always use IS NULL / IS NOT NULL.**

The slides note that **filtering, sorting, and pagination are covered in depth in the next lecture** - the material above is the "simple WHERE" foundation you need for now.

8.3 UPDATE - *correcting a mistake*

The principal walks in: *"Riya's branch was entered wrong - she's actually in CSE, not ECE."*

```
SQL
UPDATE students
SET branch = 'CSE'
WHERE name = 'Riya';
```

BEFORE		AFTER	
Riya	ECE	Riya	CSE

Always pair UPDATE with WHERE. UPDATE modifies existing data in place. Without a WHERE clause, it updates every single row in the table - a classic and costly mistake.

Syntax

```
SQL
UPDATE table_name
SET column1 = value1, column2 = value2, ...
[WHERE condition];
```

A. Updating a SINGLE attribute

```
SQL
UPDATE students
SET branch = 'CSE'
WHERE id = 2;
```

B. Updating MULTIPLE attributes at once

```
SQL
UPDATE customers
SET city      = 'Bengaluru',
    is_prime = TRUE
WHERE cust_id = 102;
```

Separate assignments with COMMAS, never AND. SET city = 'Pune' AND is_prime = TRUE is not two assignments - SQL reads it as one boolean expression and stores a wrong value. **This is the single most common UPDATE error in exams.**

C. Filtering the modification with WHERE - different rows, different filters

SQL

```
-- One specific row (primary key -> exactly one row changes)
UPDATE students SET branch = 'ME' WHERE id = 4;

-- A group of rows (all ECE students move to CSE)
UPDATE students SET branch = 'CSE' WHERE branch = 'ECE';

-- A list of rows
UPDATE students SET branch = 'CSE' WHERE id IN (1, 3, 5);

-- A range of rows
UPDATE customers SET is_prime = TRUE WHERE age BETWEEN 18 AND 25;

-- Rows matching a pattern
UPDATE customers SET city = 'Pune' WHERE email LIKE '%@pune.edu';

-- Rows with missing data
UPDATE customers SET city = 'Unknown' WHERE city IS NULL;

-- Two conditions combined
UPDATE customers
SET is_prime = TRUE
WHERE city = 'Pune' AND age > 21;
```

D. Updating using the column's current value

SQL

```
-- Everyone's age goes up by one
UPDATE customers SET age = age + 1;

-- 10% discount, only on large orders
UPDATE orders SET amount = amount * 0.9 WHERE amount > 1000;
```

The right-hand side is computed from the row's **existing** value.

E. Different values for different rows - CASE

SQL

```
UPDATE customers
SET is_prime = CASE
```

```
        WHEN age >= 25 THEN TRUE
        ELSE FALSE
    END;
```

The safe-update workflow

SQL

```
-- 1. PREVIEW: see exactly which rows will change
SELECT * FROM students WHERE branch = 'ECE';

-- 2. UPDATE with the same WHERE
UPDATE students SET branch = 'CSE' WHERE branch = 'ECE';

-- 3. VERIFY
SELECT * FROM students;

-- 4. COMMIT (or ROLLBACK if it went wrong - see Part 9)
COMMIT;
```

Golden rule: run the **SELECT** with your WHERE clause *before* you run the **UPDATE**. If the SELECT returns 500 rows and you expected 1, you just saved yourself.

8.4 DELETE - removing a student record

The principal walks in: *"This student dropped out last month. Remove their record."*

SQL

```
DELETE FROM students
WHERE name = 'Sneha';
```

Removes selected rows, with conditions. DELETE removes rows that match a WHERE condition - **one row, several, or (with no condition at all) every row** - while **the table structure stays exactly as it was**.

SQL

```
-- One row
```

```

DELETE FROM students WHERE id = 5;

-- Several rows
DELETE FROM students WHERE branch = 'ME';

-- A list
DELETE FROM students WHERE id IN (7, 8, 9);

-- Rows with missing data
DELETE FROM customers WHERE email IS NULL;

-- EVERY row (structure remains) - same effect as TRUNCATE, but
DML
DELETE FROM students;

```

Key points

- Category **DML** -> **can be rolled back** before **COMMIT**.
- Removes **whole rows only**. To blank one value, use **UPDATE ... SET col = NULL** instead.
- Blocked by foreign keys if child rows reference the row being deleted (this is integrity working correctly).

DELETE vs DROP COLUMN - don't confuse them: **DELETE FROM students WHERE id = 1;** removes a **row**. **ALTER TABLE students DROP COLUMN branch;** removes a **column**.

Quick Recall 8: DML = INSERT | SELECT | UPDATE | DELETE. INSERT without a column list -> all columns, defined order. UPDATE: commas between assignments, WHERE scopes the change. DELETE removes rows, keeps structure. SELECT changes nothing.

PART 9 - TCL: TRANSACTION CONTROL LANGUAGE

9.1 The problem: what if the system crashes mid-way?

Scenario - online banking transfer of Rs 5,000 from A to B

None

Step 1: Account A: Rs 10,000 -> Rs 5,000 (-5,000) done
SYSTEM CRASHES right at this exact moment
Step 3: Account B: Rs 8,000 -> Rs 8,000 (+0) never
ran

Money vanished from Account A and never reached Account B - unless the database can guarantee **"all steps, or none."** That guarantee is a **transaction**.

9.2 COMMIT - *locking in the changes*

"Both steps of the transfer succeeded - save it for good."

SQL

```
BEGIN;  
UPDATE accounts SET balance = balance - 5000 WHERE id = 'A';  
UPDATE accounts SET balance = balance + 5000 WHERE id = 'B';  
COMMIT;
```

What COMMIT does

- Makes every change since **BEGIN** permanent
- **Visible to every other session** once it runs
- Until then, **only this session sees it**
- **Can't be undone with ROLLBACK** afterward

Tracing the balances

	BEFORE TRANSACTION	PENDING (uncommitted)	AFTER COMMIT
A	10,000	5,000	5,000
B	8,000	13,000	13,000
Status	nothing has run yet	this session only	permanent, visible to all

Key insight: the values were **already set in the pending stage**. COMMIT only changes **visibility and permanence**.

9.3 ROLLBACK - *undoing a mistake*

"Wait - Account A has insufficient balance! Undo everything since the transaction started."

SQL

```
-- Step 1: Deduct Rs 2000 from Account A
UPDATE accounts SET balance = balance - 2000 WHERE id = 'A';

-- Step 2: Check the balance
SELECT balance FROM accounts WHERE id = 'A';
-- Suppose the query returns: -1000
-- This means Account A has insufficient balance.

ROLLBACK;
```

What ROLLBACK does

- Reverses changes since **BEGIN** / the last **COMMIT**
- **Only works before COMMIT runs**
- A safety net for mid-transaction mistakes
- After it runs, **nothing happened at all**

Tracing Account A

BEFORE	PENDING (checked)	AFTER ROLLBACK
Rs 1,000	Rs -1,000	Rs 1,000
healthy starting balance	SELECT reveals: insufficient!	original restored

The SELECT check caught it in time. Since the change was **still uncommitted**, ROLLBACK undid it instantly - **before that invalid value could ever be saved**.

9.4 SAVEPOINT - a checkpoint you return to

"This transfer has two steps - undo only the second one, not the whole transaction."

SQL

```
BEGIN;
UPDATE accounts SET balance = balance - 5000 WHERE id = 'A';
SAVEPOINT sp1;
```

```
UPDATE accounts SET balance = balance + 5000 WHERE id = 'B';
SAVEPOINT sp2;
-- step 2 was wrong, keep step 1
ROLLBACK TO sp1;
```

What SAVEPOINT does

- Marks a **checkpoint inside a transaction**
- **ROLLBACK TO** undoes **only steps after it**
- Steps **before** that checkpoint stay intact
- Lets you undo **part** of a transaction, not all of it

Tracing each checkpoint

Stage	A	B	Note
START	10,000	8,000	transaction begins
AFTER STEP 1 -> SAVEPOINT sp1	5,000	8,000	checkpoint sp1 marks this state
AFTER STEP 2 -> SAVEPOINT sp2	5,000	13,000	checkpoint sp2 marks this state
ROLLBACK TO sp1	5,000	8,000	only step 2 undone - step 1 kept

The point of SAVEPOINT: **ROLLBACK TO sp1** undoes only the credit to B - the debit from A (before sp1) stays exactly as it was. Like a save point in a video game.

Quick Recall 9: TCL = COMMIT (permanent, visible to all, irreversible) | ROLLBACK (undo since BEGIN, only before COMMIT) | SAVEPOINT + ROLLBACK TO (partial undo). Transactions guarantee **all steps, or none**.

PART 10 - DCL: DATA CONTROL LANGUAGE

DCL commands control permissions and access within a database - deciding exactly who is allowed to do what.

Real-world significance

- Faculty **should** access student records

- Interns **should not** delete student records
 - Admins get full privileges
-

10.1 GRANT - *handing out the keys*

"Give the faculty team read access to the students table - nothing more."

SQL

```
GRANT SELECT ON students TO faculty_team;
```

BEFORE

faculty_team -> students
(SELECT) **BLOCKED**

AFTER

faculty_team -> students
(SELECT) **ALLOWED**

Provides permissions. GRANT gives a user or role **specific privileges** - SELECT, INSERT, UPDATE, DELETE, or even administrative rights - on a database object.

SQL

```
GRANT SELECT, INSERT ON students TO faculty_team;  
GRANT ALL PRIVILEGES ON students TO admin_role;
```

10.2 REVOKE - *taking back access*

"The intern's project is over - remove their permission to delete records."

SQL

```
REVOKE DELETE ON students FROM intern_role;
```

BEFORE

intern_role -> students (DELETE)
ALLOWED

AFTER

intern_role -> students (DELETE)
BLOCKED

Removes permissions. REVOKE **withdraws privileges previously granted.** Together, GRANT and REVOKE let administrators keep access **tightly matched to what each role actually needs.**

Principle of least privilege: grant a role only the permissions it genuinely needs - nothing more.

Quick Recall 10: DCL = GRANT (give privileges) | REVOKE (take them back).
Least privilege. Without permission control: unauthorized access, accidental/malicious deletion, compliance violations, no accountability.

PART 11 - REVISION AIDS

11.1 Your SQL command toolkit

None

DDL	->	CREATE ALTER TRUNCATE DROP	(structure)
DML	->	INSERT SELECT UPDATE DELETE	(data)
TCL	->	COMMIT ROLLBACK SAVEPOINT	(transactions)
DCL	->	GRANT REVOKE	(permissions)

Remember the trios and pairs: DELETE vs TRUNCATE vs DROP | COMMIT vs ROLLBACK | GRANT vs REVOKE | CREATE vs ALTER

11.2 Full syntax sheet

Command	Syntax
CREATE	CREATE TABLE t (col type constraint, ...);
ALTER (add)	ALTER TABLE t ADD COLUMN c type;
ALTER (type)	ALTER TABLE t ALTER COLUMN c TYPE newtype;
ALTER (rename)	ALTER TABLE t RENAME COLUMN old TO new;
ALTER (drop)	ALTER TABLE t DROP COLUMN c;
TRUNCATE	TRUNCATE TABLE t;
DROP	DROP TABLE t;
INSERT	INSERT INTO t (cols) VALUES (vals);
INSERT (no cols)	INSERT INTO t VALUES (all vals in defined order);
SELECT	SELECT cols FROM t WHERE cond;

UPDATE	UPDATE t SET c1 = v1, c2 = v2 WHERE cond;
DELETE	DELETE FROM t WHERE cond;
GRANT	GRANT priv ON t TO role;
REVOKE	REVOKE priv ON t FROM role;
COMMIT	COMMIT;
ROLLBACK	ROLLBACK; / ROLLBACK TO sp;
SAVEPOINT	SAVEPOINT sp;

11.3 Top 12 mistakes to avoid

1. WHERE col = NULL instead of WHERE col IS NULL.
2. Forgetting WHERE in UPDATE or DELETE - the whole table changes.
3. Using AND instead of commas between SET assignments.
4. Expecting TRUNCATE ... WHERE to work - TRUNCATE takes no condition.
5. Expecting to ROLLBACK a TRUNCATE or DROP - DDL auto-commits.
6. Creating a child table before its parent (foreign key fails).
7. Inserting without a column list and getting the value order wrong.
8. Missing single quotes around text and date literals.
9. Declaring two PRIMARY KEY constraints instead of one composite key.
10. Assuming UNIQUE forbids NULLs - it doesn't; NOT NULL does that.
11. Forgetting the semicolon at the end of a statement.
12. Confusing DELETE (removes a row) with DROP COLUMN (removes a column).

11.4 Checkpoint questions from the slides

Checkpoint 1 - Data categories

1. *Is a YouTube video structured data?* **No - it's unstructured.** It has no rows/columns, only metadata tags (title, uploader, duration).
2. *Why is an Excel sheet easier to search than a folder of images?* Excel is **structured** - fixed rows/columns allow filtering and sorting. Images require **metadata-based search**, which is slower.
3. *Can all types of data be stored efficiently in tables?* **No.** Nested, variable, or media data doesn't fit rigid table schemas - that's why semi-structured and unstructured storage exists.

Checkpoint 2 - Why DBMS

1. *What happens if two Excel files contain different phone numbers for the same student?* **Data inconsistency** - there's no single source of truth. A DBMS prevents this by storing each fact once.
2. *Can multiple users safely edit the same file simultaneously?* **Not in file systems - last save wins.** A DBMS uses **locking and transactions** to handle concurrent access safely.
3. *What is an orphan record and why is it dangerous?* A record **referencing a parent that no longer exists** (e.g. an order for a deleted customer). **Foreign keys prevent this.**

Checkpoint 3 - Schema

1. *Can a primary key contain duplicate values?* **No.** A primary key must be **UNIQUE and NOT NULL** - it uniquely identifies each row.
2. *What if an order references a `customer_id` that doesn't exist?* **Referential integrity is violated** - an orphan record. The FK constraint prevents this by **rejecting the insert.**
3. *Why is "age" stored as INT instead of VARCHAR?* INT enables **math** (AVG, SUM), **range checks** (age > 18), and uses **less storage.** Datatypes enforce meaning.
4. *Candidate Key vs Primary Key?* Candidate keys are **all** attributes that CAN uniquely identify rows. The Primary Key is **the ONE chosen.**

Checkpoint 4 - DDL

1. *Which command removes only the records inside a table?* **TRUNCATE** - removes all rows instantly but keeps the table structure intact for reuse.
2. *Which command removes the complete table itself?* **DROP** - deletes the table's structure and data permanently; nothing is left to reuse.
3. *Why might ALTER become important as an application evolves?* Requirements change over time - **ALTER lets you add, modify, or drop columns and constraints without losing existing data or rebuilding the table.**

Checkpoint 5 - DML

1. *Why is the WHERE clause so important in UPDATE and DELETE?* **Without WHERE, the command applies to every row in the table** - WHERE scopes the change to only the intended row(s).
2. *Can DELETE remove only one row?* **Yes** - pair DELETE with a WHERE condition that uniquely identifies that row, e.g. `WHERE id = 5.`
3. *Which single command only retrieves data without changing it?* **SELECT** - it reads and returns rows but never modifies the underlying data.

Checkpoint 6 - TCL

1. *Why are transactions critical in financial systems?* They guarantee a multi-step operation, like a funds transfer, **either completes fully or not at all** - preventing money from vanishing if the system crashes mid-way.

2. *What happens to the changes if COMMIT is never executed?* The changes stay **pending and uncommitted** - not permanently saved, and can still be rolled back or lost if the session ends.
3. *Why might SAVEPOINT be useful in a long, multi-step workflow?* It lets you **roll back to a specific checkpoint** instead of undoing the entire transaction - like a save point in a video game.

Checkpoint 7 - DCL

1. *Should every employee get DELETE permission by default?* **No** - follow the **principle of least privilege**; grant DELETE only to roles that genuinely need it.
 2. *Why are access restrictions important in enterprise systems?* They protect sensitive data from unauthorized access or accidental/malicious changes, and keep **accountability** clear across roles.
 3. *What risks can occur without permission control?* Unauthorized data access, accidental or malicious deletion/modification, compliance violations, and no clear accountability for who changed what.
-

11.5 Practice problems

Use this schema:

SQL

```
students(id, name, branch)
customers(cust_id, name, email, age, city, is_prime)
orders(order_id, cust_id, amount, placed_at)
```

1. Create the **orders** table with **order_id** as primary key, **amount** as an exact decimal that must be positive, and **cust_id** as a foreign key to **customers**.
2. Add a **cgpa DECIMAL(4,2)** column to **students**.
3. Rename **students.branch** to **department**.
4. Change **students.name** to hold up to 100 characters.
5. Add a CHECK constraint named **chk_cgpa** allowing values from 0 to 10.
6. Insert a student **without naming the columns**.
7. Insert three students in a single statement.
8. Correct student id 3's department to **'CSE'**.
9. Set **city** to **'Pune'** and **is_prime** to TRUE for customer 105 (one statement).
10. Give everyone in Mumbai a prime membership.
11. Remove all customers who have no email recorded.
12. Show the names of all customers older than 21 who live in Pune.
13. Empty the **students** table but keep it usable for the next batch.
14. Give **faculty_team** permission to read and insert into **students**.

Answers

SQL

-- 1

```
CREATE TABLE orders (  
    order_id INT PRIMARY KEY,  
    cust_id INT,  
    amount DECIMAL(10,2) CHECK (amount > 0),  
    placed_at TIMESTAMP,  
    CONSTRAINT fk_cust FOREIGN KEY (cust_id) REFERENCES  
customers(cust_id)  
);
```

-- 2

```
ALTER TABLE students ADD COLUMN cgpa DECIMAL(4,2);
```

-- 3

```
ALTER TABLE students RENAME COLUMN branch TO department;
```

-- 4

```
ALTER TABLE students ALTER COLUMN name TYPE VARCHAR(100);
```

-- 5

```
ALTER TABLE students ADD CONSTRAINT chk_cgpa CHECK (cgpa BETWEEN  
0 AND 10);
```

-- 6 (column order: id, name, department, cgpa)

```
INSERT INTO students VALUES (13, 'Nikhil', 'CSE', 8.40);
```

-- 7

```
INSERT INTO students (id, name, department) VALUES  
(14, 'Tara', 'ECE'),  
(15, 'Yash', 'ME'),  
(16, 'Isha', 'CSE');
```

-- 8

```
UPDATE students SET department = 'CSE' WHERE id = 3;
```

-- 9

```
UPDATE customers SET city = 'Pune', is_prime = TRUE WHERE cust_id  
= 105;
```

```

-- 10
UPDATE customers SET is_prime = TRUE WHERE city = 'Mumbai';

-- 11
DELETE FROM customers WHERE email IS NULL;

-- 12
SELECT name FROM customers WHERE age > 21 AND city = 'Pune';

-- 13
TRUNCATE TABLE students;

-- 14
GRANT SELECT, INSERT ON students TO faculty_team;

```

11.6 Last-minute one-liners

- Data = raw facts. Ladder: Observations -> Data -> Information -> Insight.
- Structured = school records room | Semi-structured = university locker | Unstructured = talent hunt.
- Adding one attribute: table gives everyone NULL; JSON changes one document.
- Relational = fixed tables + shared values. Non-relational = per-record structure.
- Seven DBMS advantages: redundancy, consistency, access, concurrency, security, backup, integrity.
- Schema = blueprint with four parts: entities, attributes, relationships, constraints.
- PRIMARY KEY = UNIQUE + NOT NULL, one per table. FOREIGN KEY = referential integrity, blocks orphans.
- Superkey contains Candidate key contains Primary key.
- UNIQUE allows NULLs; NOT NULL does not.
- Four SQL families: DDL (structure) | DML (data) | TCL (transactions) | DCL (permissions).
- TRUNCATE is faster than DELETE because DELETE logs each row; TRUNCATE deallocates pages in bulk.
- DDL auto-commits - TRUNCATE and DROP cannot be rolled back; DELETE can.
- INSERT without a column list = every column, in the exact defined order.
- UPDATE: commas between assignments, never AND. No WHERE = every row.
- SELECT changes nothing. **IS NULL**, never **= NULL**.
- COMMIT makes it permanent; ROLLBACK only works before COMMIT; SAVEPOINT enables partial undo.